

🌟 Building and Flashing Oosto Yocto OS to AIMobile AIM-Edge NCOX

This guide covers building the Oosto Yocto image inside a Docker container and flashing the AIMobile AIM-Edge NCOX (Jetson Orin NX 16GB) device.

Part 1: Building with Docker

Prerequisites

- Docker installed on the host (Linux, macOS, or Windows)
- Access to [metropolis-io/yocto-oxygen](https://github.com/metropolis-io/yocto-oxygen) 🔗 Explain repository

Build the Docker image

Build from dockerfile

Get this dockerfile: [📄 Dockerfile.yocto-builder](#)

From the root of this repository:

```
1 | docker build -f Dockerfile.yocto-builder -t yocto-builder:latest .
```

The image is based on Ubuntu 24.04 and includes all Yocto build dependencies. It creates a non-root `yocto` user (required by BitBake).

(Alternative) Pulling docker image from GHCR

MacOS (ARM) users

```
1 | # Pull the specific version
2 | docker pull ghcr.io/metropolis-io/unified-yocto-builder:armv1.0
3 |
4 | # Re-tag for local convenience
5 | docker tag ghcr.io/metropolis-io/unified-yocto-builder:armv1.0 yocto-
  | builder:latest
6 |
7 | # (Optional) Untag for digital decluttering
8 | docker rmi ghcr.io/metropolis-io/unified-yocto-builder:armv1.0
```

Linux/Windows (amd64) users

```
1 | # Pull the specific version
2 | docker pull ghcr.io/metropolis-io/unified-yocto-builder:amd64
3 |
4 | # Re-tag for local convenience
5 | docker tag ghcr.io/metropolis-io/unified-yocto-builder:amd64 yocto-
  | builder:latest
6 |
7 | # (Optional) Untag for digital decluttering
8 | docker rmi ghcr.io/metropolis-io/unified-yocto-builder:amd64
```

One-time setup

```
1 | docker volume create metropolis-yocto-aimobile
```



Why Docker volumes? macOS and Windows use case-insensitive filesystems which silently corrupt Yocto builds. Docker named volumes are always case-sensitive Linux ext4.

Populate the workspace volume

The first time, start a container and clone the repository into the named volume:

```
1 docker run --name unified-yocto-builder -it \  
2   --memory=24g \  
3   --memory-swap=32g \  
4   --cpus=12 \  
5   --shm-size=4g \  
6   -v metropolis-yocto-aimobile:/home/yocto \  
7   yocto-builder:latest /bin/bash
```

Inside the container:

1. Create an ssh-key and add it to your github account

```
1 ssh-keygen -t ed25519 -C "email@metropolis.com"  
2  
3 # then add the content of ~/.ssh/id_ed25519.pub to your github ssh key  
4 # if you have gh CLI  
5 gh ssh-key add ~/.ssh/id_ed25519.pub --title "Yocto Container"
```

2. Clone the project

```
1 cd /home/yocto/  
2 git clone git@github.com:metropolis-io/yocto-oxygen.git  
3 cd yocto-oxygen  
4 git submodule update --init --recursive
```

On subsequent runs just start the container — the workspace volume persists.

Run the build

Inside the container, from the repo root:

```
1 cd /home/yocto/yocto-edge  
2 source setenv edge-aimobile-ncox oxygen-dev  
3 bitbake edge-image
```

i setenv must be *sourced*, not executed. It sets up the build directory and environment.

i NEW! We have two image types:

`edge-image-core` is the default image for production.

`edge-image-dev` is the developer version with SSH password login enabled.

The build output lands in:

```
1 build/tmp-glibc/deploy/images/edge-aimobile-ncox/
```

Key artifact for flashing:

```
1 edge-image-core-edge-aimobile-ncox.rootfs.tegraflash.tar.gz
```

Retrieving the artifact

On Host, copy the flash bundle out of the Docker volume to the host:

use `docker cp` with the container ID (or `--name` if you named the container)

```
1 docker cp <container_id>:/home/yocto/workspace/yocto-edge/build/tmp-  
glibc/deploy/images/edge-aimobile-ncox/edge-image-core-edge-aimobile-  
ncox.rootfs.tegraflash.tar.gz .
```

Alternatively, mount a host bind-mount for the deploy directory when starting the container:

```
1 docker run -it \  
2 -v yocto-workspace:/home/yocto/workspace \  
3 -v /path/on/host/deploy:/home/yocto/deploy \  
4 yocto-builder
```

Then copy the artifact there from inside the container before exiting.

Optimization: persistent downloads cache

Yocto downloads can be reused across builds by mounting a shared downloads volume. This avoids re-fetching sources on a clean build.

```
1 docker volume create yocto-downloads
```

Then add `-v yocto-downloads:/home/yocto/downloads` to all `docker run` commands above.

Part 2: Flashing the Device

Hardware

Field	Value
Product	AIMobile AIM-Edge NCOX
SoC	Tegra234 (Jetson Orin NX)
Module	p3767-0000 (16 GB)
Carrier	p3509-a02 (NVIDIA Orin NX/Nano DevKit reference)
NVMe	256 GB

Prerequisites

- Linux host with USB (the flash process uses USB device mode)

- `lsusb` available (package `usbutils`)
- the `tegraflash.tar.gz` bundle extracted

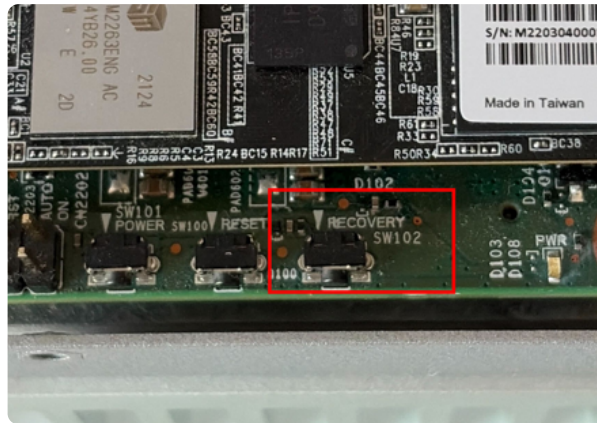
Step 1: Enter Recovery Mode

Option 1: with ssh access, run this command

```
1 | sudo /sbin/reboot --force forced-recovery
```

Option 2: with hardware access

Hold the *FORCE RECOVERY* button while power cycle the device



Note: Verify on the host

```
1 | lsusb | grep 0955:7323
```

Expect: Bus Device ID 0955:7323 NVIDIA Corp. APX

i If `0955:7323` appears, the device is in Recovery Mode and ready to flash.

Step 2: Extract the flash bundle

```
1 | mkdir -p ~/flasher
2 | tar -xzf edge-image-core-edge-aimobile-ncox.rootfs.tegraflash.tar.gz -C
   | ~/flasher/
3 | cd ~/flasher
```

i Change to `edge-image-dev` if you built the dev image

Step 3: Disable host auto-mounting

`initrd-flash` exposes the NVMe as USB mass storage during the flash process. If the host automounts it, `partprobe` fails with: Device `/dev/sda1` is mounted (`udisks-error-quark, 14`)

Prevent this:

```
1 | sudo systemctl stop udisks2
```

Step 4: Flash

```
1 | sudo ./initrd-flash
```

`initrd-flash` boots a minimal initramfs on the device over USB, then writes:

- QSPI bootloader partitions
- NVMe rootfs, docker images volume, and data volume

⚠ This is the correct tool. Do *not* use `doflash.sh` (QSPI only) or `doexternal.sh` (broken — APPFILE placeholder bug).

The flash takes several minutes. When complete the device reboots automatically.

Step 5: Re-enable automounting

```
1 | sudo systemctl start udisks2
```

Default credentials

User	Password	Notes
root	metro12345	
metro	metro12345	<code>sudo</code> with no password required

Partition layout (NVMe)

Partition	Size	Purpose
A_kernel	128 MB	Kernel slot A
A_kernel-dtb	—	DTB slot A
A_reserved	—	Reserved slot A
B_kernel / B_kernel-dtb / B_reserved	same	Kernel slot B (A/B redundancy)
recovery	—	Recovery kernel
esp (×2)	64 MB each	UEFI ESP
reserved	480 MB	Reserved

APP	8 GB	Root filesystem slot A
APP_b	8 GB	Root filesystem slot B
DOCKER	28 GB	Docker images volume
DATA	~74 GB	User data volume

Troubleshooting

Device not seen in Recovery Mode

- Confirm the USB cable is connected to the *micro-USB Recovery port* (not the standard USB-A ports).
- Re-try the button sequence: hold FORCE_RECOVERY first, then power-cycle.